Hackaton Build with AI

Общая информация

Название хакатона	Build with AI
Организатор	Школа ИИ и науки о данных
Партнёры задачи	НИЦ Smart City, GDG Astana

1) Задача от НИЦ Smart City

«ИИ-помощник по прогулкам в городе Астана»
Кратко:
На входе — набор данных по точкам интереса города (POI) с короткими описаниями. Задача команды — придумать и показать рабочую *Retrieval-Augmented Generation (RAG)* систему (концепцию + демо), которая в реальном времени подсказывает пользователю **куда сходить и что сделать** в зависимости от его текущей локации, предпочтений и контекста.
Цель
Создать полезный, безопасный и удобный городской советчик, который сочетает поиск по базе POI и генерацию человекоподобных рекомендаций — чтобы пользователь быстро получил релевантные, конкретные и вдохновляющие подсказки о том, как провести время рядом с собой.
Что ожидается от команд
1. **Концепция продукта** — краткое описание идеи, целевой аудитории, сценариев использования.
2. **Демо / прототип** — интерактивная демонстрация поведения системы на нескольких сценариях (реальные или смоделированные локации и запросы). Демо может быть как UI-прототипом, так и диалоговой демонстрацией (например, чат).
3. **Описание работы RAG-подхода (высокоуровневое)** — как система использует данные POI + генерацию для выдачи рекомендаций (не нужны низкоуровневые архитектурные схемы).
4. **Набор тестовых сценариев** — несколько реальных пользовательских кейсов и примеры входных данных + ожидаемый тип ответа.
5. **Краткая презентация** (5–7 минут) с бизнес-идей, ограничениями, рисками и планом развития.
6. **Этические и privacy-заметки** — как команда учитывает конфиденциальность локации, борьбу с недостоверной информацией и безопасность пользователей.

Данные
- Название точки (например, «Кафе «Коктем»»).
- Координаты (ориентировочно — для сопоставления с локацией пользователя).
- Короткое описание.
- Категория / теги (кафе, музей, парк, смотровая площадка и т.д.).
- *Время работы / базовые контактные данные (если есть).*
- *Рейтинг / популярность (если доступно).*

Командам нужно заниматься сбором данных — датасет должен быть собран и процесс сбора должен быть автоматизирован участниками хакатона.
Пользовательские сценарии (примерные)
- «Я сейчас возле Mega Silk Way, хочу прогуляться 30–60 минут — что посоветуете?»

- «Хочу тихое место с кофе и розеткой недалеко от меня.»
- «Еду с ребёнком — где рядом есть детские площадки или музей с интерактивами?»
- «Я - турист, у меня 2 часа перед вылетом: что можно посмотреть поблизости?»
- «Ищу необычную ночную активность (последние пару часов).»

Для каждого сценария ожидается выдача: 1–3 конкретных пункта с кратким объяснением **почему** (короткая мотивация), примерным временем на посещение и опцией «что делать там» (конкретное действие/маршрут/рекомендация).

Формат сдачи

- Ссылка на репозиторий (код/скрипты — опционально), либо архив с прототипом.
- Презентация (PDF/слайды) — 5–7 минут.
- Документ с описанием тестовых сценариев и ограничений.
- Загруженное демо на Youtube и доступ по ссылке

Критерии оценки (в порядке важности)

1. **Релевантность и полезность рекомендаций** — насколько ответы соответствуют локации, запросу и ожиданиям пользователя.
2. **Контекстуальность и адаптивность** — учёт расстояния, времени, категории пользователя и явных ограничений (время в пути, режим работы).
3. **Качество объяснений** — краткое «почему» для каждой рекомендации (помогает доверию).
4. **UX / понятность демо** — насколько просто протестировать систему и получить полезный результат.
5. **Креативность и уникальные фишки** — специальные фильтры, сценарии, игровые элементы, интеграции (например, микро-маршруты).
6. **Этика и приватность** — минимизация хранения точных координат пользователей, прозрачность работы с данными, работа с недостоверной информацией.
7. **Степень завершённости** — полнота документации, тестовых сценариев и качества презентации.

Каждый критерий оценивается жюри по шкале (например, 1–10) с суммарным весом по положению организаторов хакатона.

Ограничения и требования (высокоуровневые)

- Система должна **ясно указывать**, какие данные использует (локация пользователя, фильтры, теги).
- Избегать уверенных утверждений о доступности места (например, «открыто сейчас»), если данных нет — указывать степень уверенности.
- Рекомендации должны быть честно помечены (источник: данные POI vs. сгенерированная идея).

Примеры ожидаемого вывода (кратко, человеческим языком)

Вход: «Я у здании Экспо 2017, хочу что-то необычное на 1–2 часа, пешком.»

Вывод:

1. *Музей будущего искусства (300 м, 5 мин пешком)*. Почему: интересные интерактивные экспозиции + сейчас идёт вечерняя выставка; план действий: 40–60 мин на экспозицию, затем кофе в подсобном кафе рядом.
2. *Скрытая смотровая - «Лофт на крыше отеля Хилтон» (450 м)*. Почему: авторская терраса с видом, рекомендовано для фотосессии; план: 20–30 минут, лучше заходить при закрытой погоде.

Рекомендации для команд (опционально)

- Сфокусируйтесь на **двух–трёх** сильных пользовательских сценариях, вместо попытки покрыть всё.
- Отдайте приоритет ясности ответов пользователю - короткие, конкретные рекомендации лучше длинных рассуждений.

- Пропишите, как система будет обрабатывать сомнительные или неполные данные (например, закрытые места, временные мероприятия).

Стретч-цели (для баллов «креативность»)

- Поддержка групповых рекомендаций и планирования (что делать для группы из 3–5 человек).
- Генерация мини-маршрута (последовательность 3 мест с маршрутом и временем).
- Мультиязычность (советы на языке пользователя).
- Интеграция с локальными событиями (временные выставки, вечеринки).
- Доступность (рекомендации для людей с ограниченной подвижностью).

2) Задача от GDG Astana

Case: CampusPulse AI — Smart Autonomous AI Agent with MCP Support

Case Goal

Students frequently face challenges with managing their academic schedule, tracking assignment deadlines, and retrieving up-to-date information.

The goal for each team is to develop an autonomous AI Agent that helps students organize their coursework, plan tasks, and interact with external services. Instead of using static pre-built datasets, the agent must interact with the real world using the **MCP (Model Context Protocol)** and custom Tools (Function Calling), and then be deployed to **Google Cloud Platform (Vertex AI Agent Engine / Reasoning Engine)**.

Key Requirements & Objectives

Each team is free to define their own CampusPulse AI concept and toolset.

For example, the agent can:

- Structure the academic week and calculate assignment deadlines per course,
- Integrate with local files (lecture notes, PDFs) to analyze study materials,
- Search for real-time information regarding events, schedules, and campus locations via external APIs,
- Interact with GitHub to check assignment repositories and tasks,
- Utilize custom MCP servers to automate routine student tasks.

Key Focus: Demonstrate how the agent executes its decision-making process (Reasoning Loop), invokes the appropriate tools via MCP / Function Calling, and effectively solves real student problems.

Core Deliverables

Teams must build a functional prototype demonstrating the use of Google Cloud services to deploy and serve the AI Agent. The project should implement the following components:

1. **Agent Core & MCP Module (AI Agent & Tools Core):** Build the agent logic using Gemini API (via Vertex AI SDK, LangChain, or LlamaIndex). Integrate at least 1–2 Tools or an MCP (Model Context Protocol) server so the agent can query external data or execute actions.
2. **Cloud Deployment (GCP Agent Engine / Reasoning Engine):** Deploy the created agent to **Vertex AI Reasoning Engine**

(reasoning_engines.ReasoningEngine.create), turning it into a cloud-hosted service with an accessible API.

3. **User Interface (Frontend / Demo Client):** Build a minimal web interface (Streamlit, Gradio, or a Cloud Run hosted app) to submit user queries to the agent and visualize its reasoning steps and tool calls.
4. **Solution Demo & Presentation:** Prepare a project presentation covering the system architecture, MCP tool execution, GCP deployment process, and the overall value provided to students.

Recommended Google Cloud & MCP Tech Stack

Teams are encouraged to utilize Google Cloud services and the MCP framework within the following stack:

Purpose	Tool / Technology
Agent Core & LLM	Vertex AI Gemini API (Gemini 1.5 Flash / Pro)
Orchestration & MCP	Vertex AI SDK / FastMCP / MCP Python SDK / LangChain
Agent Deployment & Hosting	Vertex AI Reasoning Engine (Agent Engine)
UI Hosting	Cloud Run / Firebase Hosting
Session State & Storage	Firebase Firestore / Cloud SQL
Sample MCPs / Tools	Google Maps MCP, GitHub MCP, FileSystem MCP, Custom MCP

Evaluation Criteria

Criterion	What is Evaluated
1. MCP & Tools Usage	How effectively and correctly the agent uses MCP servers and tools (Function Calling) to solve tasks.
2. GCP Agent Engine Deployment	Successful deployment to Vertex AI Reasoning Engine and API availability.
3. AI Logic & Autonomy	Quality of the reasoning chain (Reasoning Loop), logical tool selection, and response accuracy.
4. UX & Visualization	Usability of the demo UI and clear visualization of the agent's internal steps.
5. Architecture & Presentation	Code quality, repository structure, documentation of MCP tools, and project presentation.

Submission Requirements

Each team must submit:

1. **Repository Link (GitHub / GitLab):** Source code of the agent, MCP configuration, Reasoning Engine deployment script, and setup instructions in README.md.
2. **Project Presentation (PDF or PPTX format**).

Good luck!